BEGINNER'S GUIDE · AI ALGORITHMS

Understanding AI in Face Recognition

A beginner-friendly explanation of every AI algorithm used in this face payment system — what it is, why we need it, how it works, and real-world examples.

1. **Haar Cascade (Viola-Jones)**
   Face Detection — Where is the face?

2. **KNN + Euclidean Distance**
   Face Recognition — Whose face is this?

3. **DeepFace MiniFASNet**
   Anti-Spoofing — Is this a real face?

ALGORITHM 1 OF 3

Haar Cascade

Also called Viola-Jones Algorithm · Classical Computer Vision · Invented 2001

❓ WHY DO WE NEED IT?

Before you can recognize someone's face, you first need to **find where the face is** in the image. A camera photo is just a big grid of pixels. The computer doesn't automatically know which pixels belong to a face.

Imagine trying to find a cat in a 1000-piece puzzle. You first need to *locate* the cat area before you can study its features. Haar Cascade solves exactly this: it scans the entire image and draws a box around every face it finds.

> **REAL-WORLD ANALOGY**
>
> Think of a security guard scanning a crowd. Before checking IDs, they first *spot* each person's face. Haar Cascade is the "spotting" step — it points to where faces are, nothing more.

📖 WHAT IS IT?

Haar Cascade is a **trained classifier** — a file that learned from thousands of face and non-face images. It uses simple rectangular patterns called **Haar-like features** to detect faces.

These features look at differences in brightness between adjacent rectangles in an image. For example, the eye region is usually darker than the cheek below it — this brightness pattern is a "Haar feature."

The **cascade** part means the classifier is made of many stages — early stages quickly reject areas that obviously aren't faces, so the algorithm is very fast.

⚙️ HOW DOES IT WORK? (STEP BY STEP)

1 **Convert to Grayscale**

Color images are converted to grayscale. Face detection only needs brightness information, not color. This makes it 3× faster.

2 **Sliding Window Scan**

A small window (e.g. 24×24 px) slides across every position in the image. At each position, it checks: "Does this patch look like a face?"

3 **Compute Haar Features**

For each window position, it computes brightness differences between rectangles (forehead vs eyes, nose bridge vs cheeks etc.). Uses an **Integral Image** trick to compute these instantly.

4 **Cascade of Classifiers (Fast Rejection)**

The cascade has ~20 stages. Stage 1 uses just 2 features and rejects 50% of windows instantly. Only windows that pass ALL stages are declared a face. This is why it's fast — 99% of windows are rejected early.

5 **Scale the Window (Multi-Scale)**

The window rescales and repeats to detect faces at different sizes. `scaleFactor=1.3` means the window grows by 30% each pass.

6 **Return Bounding Boxes**

All detected face regions are returned as `(x, y, width, height)` rectangles. In this app: take the largest face, crop it, resize to 100×100 px.

```
# Actual code from this app gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) faces = cascade.detectMultiScale(gray, scaleFactor=1.3, # window grows 30% each pass minNeighbors=5 # need 5 overlapping detections → less false positives ) x, y, w, h = max(faces, key=lambda f: f[2]*f[3]) # largest face crop = frame[y:y+h, x:x+w] face = cv2.resize(crop, (100, 100))
```

HOW TO CHOOSE THE PARAMETERS?

PARAMETER	VALUE USED	LOWER VALUE	HIGHER VALUE
`scaleFactor`	**1.3**	Detects more face sizes but **slower**	Faster but may **miss faces** at certain sizes
`minNeighbors`	**5**	More detections but **more false positives** (detects non-	Fewer false positives but may **miss real**

PARAMETER	VALUE USED	LOWER VALUE	HIGHER VALUE
		faces)	**faces**

RULE OF THUMB

Start with `scaleFactor=1.1` and `minNeighbors=3` for high recall. Increase `minNeighbors` to 5–7 if you get false detections. Increase `scaleFactor` to 1.3 if speed is more important than catching all sizes.

✅ WHEN SHOULD YOU USE HAAR CASCADE?

Use it when: you need real-time face detection, you don't have a GPU, the faces are roughly frontal, and you want something easy to set up.

Avoid it when: faces are tilted/sideways, you need very high accuracy, or faces are very small in the image. For those cases, use **MTCNN** or **RetinaFace** (deep learning detectors).

🌍 REAL-WORLD USE CASES

📷

Camera Auto-Focus

Your phone camera detects and focuses on faces automatically using Haar Cascade or similar.

🏭

Attendance System

Office/school cameras detect employee or student faces in real-time for automated check-in.

📱

Face Filters (Snapchat)

AR filters first detect where your face is, then overlay the filter on the correct position.

🔒

This App

Crops the face region from webcam frames before enrollment and before identity matching.

ALGORITHM 2 OF 3

K-Nearest Neighbors (KNN)

Face Recognition using Euclidean Distance · Classical Machine Learning

? WHY DO WE NEED IT?

After Haar Cascade detects and crops the face, we have a 100×100 pixel image. Now the question is: **"Is this the same person who enrolled?"**

We need an algorithm that can compare the current face against stored faces and decide: same person or different person? KNN solves this by measuring how *similar* (close) the new face is to previously stored faces.

> **REAL-WORLD ANALOGY**
>
> Imagine you have 20 photos of your friend Alice. A stranger shows up. You compare the stranger to all 20 photos and check: does this stranger look like Alice? If the 5 most similar comparisons all say "yes, very close", you conclude it's Alice. That's exactly what KNN does — but with pixel numbers instead of visual intuition.

📖 WHAT IS IT?

KNN (K-Nearest Neighbors) is one of the **simplest machine learning algorithms**. It works on a basic idea: *things that are similar tend to be close together*.

Instead of "learning" a complex model, KNN just **remembers all training examples** and, when given a new input, finds the K most similar stored examples and makes a decision based on them.

In this app, each face is converted to a **30,000-dimensional vector** (100×100×3 color channels = 30,000 numbers). KNN measures the Euclidean distance between the new face vector and all stored face vectors.

📐 THE CORE FORMULA — EUCLIDEAN DISTANCE

Euclidean distance is the straight-line distance between two points. You already know it from geometry (Pythagoras theorem). Here we apply it to 30,000 dimensions instead of 2D.

$$d(A, B) = \sqrt{\left(\Sigma \ (A_i - B_i)^2 \right)}$$

for i = 1 to 30,000 dimensions

A = probe face vector (new photo) | B = enrolled face vector (stored)

d = distance (smaller = more similar)

Simple 2D example: Point A = (2, 3), Point B = (5, 7) → d = $\sqrt{((5-2)^2 + (7-3)^2)} = \sqrt{(9+16)} = \sqrt{25} = 5$. KNN does the same across 30,000 numbers.

```
# Actual code from this app def calc_distance(v1, v2): return
np.sqrt(((v1 - v2) ** 2).sum()) # Compare probe to all N enrolled faces
distances = [calc_distance(probe, enrolled[i]) for i in range(N)] #
Take 5 nearest, average their distances avg_dist =
np.mean(sorted(distances)[:5]) # Decision: is it the same person?
matched = avg_dist < 10000
```

HOW DO WE CHOOSE K? (THE MOST COMMON QUESTION)

K is the number of nearest neighbors we look at before making a decision. Choosing K is a trade-off:

K VALUE	BEHAVIOR	PROBLEM
`K = 1`	Uses only the single closest face	Very sensitive to noise — one bad photo ruins it
`K = 5` ✓ **used here**	Averages 5 closest faces	Good balance — noise-resistant, still accurate
`K = 20`	Averages many faces	Too smooth — may wrongly accept different people

RULES OF THUMB FOR CHOOSING K

1. **Use odd K** to avoid ties in classification voting.
2. **Start with K = √N** where N = number of training samples (e.g. 20 samples → K ≈ 4 or 5).
3. **Try K = 1, 3, 5, 7** on your data and pick the one with lowest error.

In this app: K=5 because each user enrolls ~20 frames, so K=5 is about 25% of the data — a good balance.

🎯 HOW TO CHOOSE THE DISTANCE THRESHOLD (10,000)?

After computing `avg_dist`, we need to decide: below what value is it a "match"? This is the **threshold**.

SAME PERSON	THRESHOLD → 10,000	DIFFERENT PERSON
avg_dist ≈ 4,000 – 8,000	Decision boundary	avg_dist ≈ 12,000 – 25,000
pixels look very similar	tuned by testing	pixels look different

The threshold of 10,000 was determined by testing real enrollments and verifications. In real logs: `avg_dist=6576` → correct match. You would tune this by testing with multiple people and finding the value that minimizes both false accepts and false rejects.

✅ WHEN SHOULD YOU USE KNN?

Use KNN when: your dataset is small, you want no training phase, you need a simple interpretable result, or you're building a prototype quickly.

Avoid KNN when: your dataset is very large (slow at prediction), features aren't meaningful raw pixels (need better embeddings), or you need very high accuracy across many users.

🌍 REAL-WORLD USE CASES

🎵

Music Recommendation

Spotify finds songs "nearest" to what you've listened to before.

🏥

Medical Diagnosis

Given patient symptoms, find the K most similar previous patients and use their diagnosis.

📧

Spam Detection

💳

This App

New email is compared to known spam/non-spam examples. K closest emails decide if it's spam.

Compare checkout face against all stored face vectors. If avg of 5 nearest < 10,000 → payment approved.

ALGORITHM 3 OF 3

DeepFace MiniFASNet

Passive Liveness Detection · Anti-Spoofing · Deep Learning (CNN)

? WHY DO WE NEED IT?

Imagine KNN perfectly identifies Alice's face. But what if a thief simply **holds up a photo of Alice** in front of the camera? KNN would still say "match = true" — because the photo looks like Alice.

This is called a **spoofing attack**. Without anti-spoofing, the entire face recognition system can be defeated with a single printed photo.

> **REAL-WORLD ANALOGY**
>
> A nightclub bouncer checks IDs but also makes sure the person is actually there in front of them — they don't accept a photocopy of an ID. MiniFASNet is the bouncer checking that you're a real, live person — not a photo, video recording, or 3D mask.

WITHOUT ANTI-SPOOFING

Someone could steal your photo from social media and hold it up to the camera to authorize a payment in your name. This is why anti-spoofing **must run before** identity matching.

📖 WHAT IS IT?

DeepFace is an open-source Python library from Meta (Facebook) that wraps multiple face AI models. It provides face detection, recognition, and anti-spoofing in one package.

MiniFASNet (Mini Face Anti-Spoofing Network) is a **Convolutional Neural Network (CNN)** specifically trained to distinguish real faces from fake ones. It was trained on large datasets of both live faces and spoof attacks (printed photos, screen replays, silicone masks).

It is called *passive* liveness detection because the user doesn't need to do anything special (no blinking, no head turning) — just look at the camera normally.

🧠 WHAT IS A CNN? (THE TECH BEHIND MINIFASNET)

A CNN (Convolutional Neural Network) is a type of deep learning model designed to process images. It automatically learns patterns from training data — you don't program the rules manually.

1. **Convolutional Layers — Feature Detection**
   Small filters slide over the image and detect low-level features (edges, textures, color gradients). Fake faces from a screen have different pixel textures than real skin.

2. **Pooling Layers — Reduce Size**
   Downsamples the feature maps to reduce computation and keep only the most important patterns.

3. **Fully Connected Layers — Decision**
   The extracted features are flattened and passed through dense layers that output a score: how likely is this a real face?

4. **Output: is_real + antispoof_score**
   `is_real = True/False` · `antispoof_score` = confidence 0.0 to 1.0

```
# Actual code from this app result = DeepFace.extract_faces(
img_path=frame, anti_spoofing=True, enforce_detection=False ) is_live =
result[0].get("is_real", False) spoof_score =
result[0].get("antispoof_score", 0.0) if not is_live: return { "match":
False, "reason": "liveness_failed" }
```

⚠️ WHAT SPOOF ATTACKS DOES IT DETECT?

🖼️

Print Attack

Holding a printed photo in front of the camera. Most common and easiest attack.

📺

Replay Attack

Playing a video of the victim's face on a phone/screen in front of the camera.

🎭

3D Mask Attack

Wearing a 3D printed or silicone mask of the victim's face.

✅

Real Face

A live human face has skin texture, micro-movements, and depth cues that a CNN can detect.

✅ WHEN SHOULD YOU USE ANTI-SPOOFING?

Always use it when: face recognition is used for security-sensitive actions — payments, access control, authentication, attendance, or any system where impersonation causes real harm.

Can skip when: it's just a fun app (e.g. face filter, demo), the environment is fully controlled, or you accept the security trade-off.

🌍 REAL-WORLD USE CASES

🏦

Mobile Banking

Apple Face ID, Samsung biometrics — all use liveness detection before authorizing payments.

🚪

Smart Door Lock

A door lock that opens by face recognition must ensure someone can't open it with a photo.

🏛️

KYC / eID Verification

Online bank account opening uses liveness checks to verify the applicant is physically present.

💳

This App

Runs first during every checkout. Spoof detected → payment blocked immediately, no KNN check needed.

SUMMARY

All Three Algorithms Together

How they work in sequence during a payment transaction

📸 **User looks at camera & clicks "Confirm & Pay"**
1 JPEG frame captured from webcam

STEP A **Haar Cascade — Find the Face**
Converts to grayscale → scans entire image → finds face bounding box → crops → resizes to 100×100 px → flattens to 30,000-dim vector

OUTPUT
100×100 crop
30K vector

STEP B **MiniFASNet — Is This a Real Face?**
CNN analyzes skin texture + micro-details → returns is_real (True/False) + confidence score

IF FAIL
STOP ❌
Spoof detected

STEP C **KNN — Is This the Right Person?**
Load enrolled vectors → compute Euclidean distance to each → sort → average top-5 nearest → compare to threshold 10,000

IF FAIL
STOP ❌
No match

✅ **Both checks passed → Payment Approved**
Balance deducted, order created, receipt shown

#	ALGORITHM	TYPE	QUESTION IT ANSWERS	KEY PARAMETER
1	**Haar Cascade**	Classical CV	Where is the face?	`scaleFactor=1.3`, `minNeighbors=5`
2	**DeepFace MiniFASNet**	Deep Learning CNN	Is this a real face?	Pretrained model, no tuning needed
3	**KNN + Euclidean**	Classical ML	Is this the right person?	`K=5`, `threshold=10,000`

📚 SUGGESTED LEARNING PATH

Beginner (start here)

→ Python basics + NumPy arrays
→ What is a pixel? RGB vs BGR
→ Euclidean distance (Pythagoras)
→ OpenCV basics (imread, resize)
→ Haar Cascade tutorial

Intermediate (go deeper)

→ What is a Neural Network?
→ CNN architecture (conv + pool + FC)
→ What is a feature vector / embedding?
→ PCA for dimensionality reduction
→ FaceNet / ArcFace (deep embeddings)
